

面试押题 02: Sink FC1 计算与通信竞争

对应九坤实习：Sink FC1 计算与通信竞争优化。核对日期：2026-09-29。本文以 2026-09-04 修正后的 M=8192 结论为准，旧 M=4096 扫描不能作为生产推荐。

0. 开源仓库技术介绍与应用

Transformer Engine 提供 Transformer 层、低精度计算及相关并行能力；DeepGEMM 提供 GPU GEMM 实现，优化矩阵分块、数据搬运和计算调度。前者是框架与层的接入位置，后者是本实验替换矩阵乘的候选库。

典型应用：大模型训练的线性层、MoE 专家矩阵乘及低精度计算。本题关注 GEMM 与通信并发时的资源分配：单算子最快不一定联合时间最短；我的贡献是定位真实 shape、构建复现并联合调参，不是原创这些库。

1. 一分钟口述 (STAR)

背景 S：训练 trace 中 Sink FC1 接近 200 μ s，最初按函数入口 shape 计算时看起来吞吐只有预期一半，同时不同实例还存在 30-60 μ s 的额外延迟。

任务 T：我负责判断是真实算子退化、shape 口径错误，还是计算与通信争抢资源，再给出能复现的调优方案。

行动 A：我沿 Transformer Engine 源码与 Nsys range 发现，入口 [4096, 1, 3072] 会先做 TP AllGather，实际 GEMM 是 8192 $\times$ 3072 $\times$ 3072，因此约 200 μ s 基线是正常的；随后逐实例关联其他 stream 的 NCCL，确认 overlap 带来真实退化，再搭建 4 $\times$ H200 双 stream 代理，联合扫描通信 CTA 与 DeepGEMM persistent SM budget，完成 121 组粗扫、89 组细扫及独立复验。

结果 R：正确 shape 下，AllGatherV 的联合 span 从 328.160 μ s 降到 286.144 μ s，降低 12.80%；与同一通信设置的 torch 对照相比，DeepGEMM 控核独立贡献 7.58%。这是四卡代理中的联合时间，不是已经证明完整训练 step 提升 12.80%。

2. 系统在做什么，我负责什么

Sink FC1 是训练中一条线性计算路径。其 `_Linear` 入口记录的是 sequence-parallel gather 前的输入，实际 GEMM 消费 gather 后的张量。本人负责 trace 与源码映射、实际 shape/FLOPs 纠错、同版本算子复现、NCCL 竞争分组统计，以及双 stream 代理和联合预算扫描。

本项不是修改 NCCL 通信协议，也不是把 GPU 硬切成两块；主要是在既有通信与 DeepGEMM 能力下控制并行工作规模，找出适合 overlap 的联合运行点。

3. 首先纠正“错误问题”，再优化真实问题

3.1 为什么最初看起来 FC1 慢了一倍

实际执行链是：`_Linear` 输入 [4096, 1, 3072] $\rightarrow$ TP AllGather [8192, 1, 3072] $\rightarrow$ GEMM M/N/K=8192/3072/3072。

GEMM FLOPs 近似为 2MNK。漏掉 AllGather 会把 M 少算一半；修正后计算量为 154.619 GFLOP，200.097 μ s 对应 772.72 TFLOP/s。FC2 的工作量约为一半，101.952 μ s 对应 758.29 TFLOP/s，两个算子处于相近性能档。

这不是猜测：原 trace 的 1,440 个目标 FC1 range 全部匹配到同一 2 $\times$ AllGather；生产同 commit 的 TE `linear.py` 先把 `inputmat_total` 更新为 gather 输出，再传给 `general_gemm`。本地相同生产 TE/cuBLAS 的 M8192 基线为 192.464 μ s，与生产相差 3.81%，足以否定“FC1 固有 2 $\times$ 异常”的说法。

3.2 真正的退化来自其他 stream

FC1 自身前置 TP AllGather 与 FC1 有数据依赖、在同一计算 stream 串行，不是本项竞争对象。竞争来自其他通信 stream 上尚未完成的 MoE SendRecv/AllGatherV。

原始生产 Sink FC1 分组	样本数	FC1 中位数	相对无跨流通信
无跨流 NCCL	1,134	199.744 μ s	基线
与 SendRecv 重叠	286	230.657 μ s	+15.48%
与 AllGatherV 重叠	20	257.137 μ s	+28.73%

这些数字说明 overlap 和延迟相关；本地控制实验在正确 shape 下复现 baseline 及相似 overlap 退化，进一步支持资源竞争解释。不能仅凭时间线重叠就断言全部是 SM 原因，因为 L2、HBM 和片上互联也可能竞争。

4. 实现步骤、目标函数和取舍

1. **建立逐实例链路**。将 CPU/NVTX range、CUDA Runtime 调用和 GPU kernel 关联，核对真正的 GEMM 输入；区分 Sink FC1 与其他相同 M8192 的 attention Linear，不能混合样本。
2. **匹配基线**。使用生产 TE 与 cuBLAS 二进制，验证 M8192 和 FC2 的 kernel 名、grid/block 与延迟；把 CUDA Event 整段时间与 Nsys 纯 kernel 时间分开。
3. **搭建双 stream 代理**。一条 stream 做 FC1，一条发起变长通信；使用独立 rank 输入、与生产量级相近的载荷，先确认真实 overlap，再谈性能。
4. **联合预算扫描**。`deep_gemm.set_num_sms(x)` 控制 persistent CTA 规模；`NCCL_MIN_CTAS/MAX_CTAS` 请求通信并行度。对角线和非对角线都测，而不是硬套“两个数字相加等于 132”。
5. **以联合 span 优化**。计算 `max(通信结束, FC1结束) - min(通信开始, FC1开始)`；每轮取四个 rank 最慢值，观察 median、P95、FC1/通信分项及实际重叠。
6. **消融与复验**。除了总基线，还与相同 NCCL 设置的 torch 比较，隔离 DeepGEMM 的独立贡献；候选重新运行，检查实际 kernel grid 而非只记录请求值。

取舍要点：减少计算 CTA 可能让孤立 GEMM 变慢，却让其与通信的联合时间变短；反之，把资源尽量给通信也不保证联合最优。目标是训练依赖链上两项都完成的时间，而不是某一颗 kernel 单独最漂亮的数字。

5. 实验矩阵与结果

5.1 本地代理配置

项目	口径
硬件	4×H200，每卡 132 SM
软件	PyTorch 2.10.0+cu128、CUDA 12.8、NCCL 2.27.5、DeepGEMM 2.6.1
FC1	BF16, M/N/K=8192/3072/3072
AllToAllV	每 rank 24 MiB, 1:2:3:4 不等长 split
AllGatherV	平均每 rank 6 MiB, 1:2:3:4 不等长输入
并发	双 stream, CUDA_DEVICE_MAX_CONNECTIONS=8
统计	10 次预热、20 次测量；每轮四 rank 最大值后求 median/P95
搜索规模	121 组粗扫、89 组细扫、15 个独立复验候选

5.2 正确 M8192 下的候选

场景	代理候选	联合 span median / P95	总降幅	同通信设置下 DG 独立降幅
AllGatherV	请求/实际 12 CTA + DG120	286.144 / 316.622 μ s	12.80%	7.58%
AllToAllV, median 优先	请求 24 CTA、实际 grid 32 + DG112	308.226 / 424.779 μ s	81.04%	1.67%
AllToAllV, 尾延迟优先	同通信设置 + DG96	311.616 / 330.018 μ s	不以此行给新 headline	相对最优 median 仅慢 1.10%

AllGatherV 总降幅采用同次粗扫的 8 CTA + torch132、328.160 μ s 为基线；7.58% 则固定为同 12-CTA 通信设置后比较，是不同分母。两轮复验中，12 CTA + DG120 的 160/160 个 rank 样本真实重叠。

AllToAllV 的 81.04% 主要来自通信实际并行度从 2 CTA 提升到 32 blocks，不能声称是 DeepGEMM 控核带来 81%；同通信设置下独立贡献只有 1.67%。DG96 比 DG112 的 P95 低 94.761 μ s，体现 median 与尾延迟的取舍。

5.3 历史结果如何处理

早期 M4096 代理得出 16+116、24+108 等点，以及 AllGatherV 19.29% 的改善，实验本身可以说明趋势，但计算量少一半，不能拿来替代最终 M8192 推荐。简历采用 12.80%/7.58% 正是纠正后结果。

6. 高频问答（10 题）

Q1：你做的到底是 bug 修复还是性能优化？

两部分。先修正分析口径：200 μ s 基线不是 bug，错误在于把 gather 前 shape 当成 GEMM shape。随后优化真实的跨 stream 竞争：同一个 GEMM 与 NCCL 并发时变慢，通过联合并行度调节缩短两者共同完成时间。

Q2：为什么相信是通信竞争，而不是不同输入导致？

生产先按同一 FC1 路径和真实 shape 分组，再按跨 stream NCCL overlap 分类；本地固定 M8192、相同 kernel/输入量，分别无通信、SendRecv 代理、GatherV 代理，复现了相近退化。仍保留代理实现与生产通信栈不同的边界，而不把相关性当作所有微观竞争来源的证明。

Q3：NCCL CTA 数就是它独占的 SM 数吗？

不是。CTA 是工作块数量，硬件调度、资源需求及共驻留决定实际 SM 使用；某些通信请求还会被映射到不同实际 grid。本例请求 24 CTA 的 A2A 实际 grid.x=32，所以必须从 Nsys 核对。DG SM budget 同样不等于硬件隔离，L2/HBM 仍然共享。

Q4：为什么不能让两者预算直接相加等于 132？

这只能是搜索启发式，不能是正确性或最优性定理。GEMM block 与 NCCL block 的寄存器、共享内存、执行行为不同，通信实现还会调整 grid。我们同时扫对角线和非对角线，以实际联合 span 与 P95 决定候选。

Q5：为什么不用 FC1 latency 做最终目标？

训练要等计算和必要通信都完成。少给 NCCL 资源可能让 FC1 更快、通信却长很多；只压 FC1 无法保证下游更早开始。联合 span 更接近局部依赖链目标，但完整训练仍要看关键路径和 step time。

Q6: 为什么统计四个 rank 中最慢的那个?

通信/训练同步由慢 rank 约束, rank 平均会掩盖尾部。每轮先取四 rank 最大联合时间, 再对轮次取 median/P95; 同时保留逐 rank 和真实重叠统计, 防止“有的 rank 没并发”制造虚假收益。

Q7: 12.80% 和 7.58% 有什么关系, 能相加吗?

不能。12.80% 是最终组合对原通信+torch 基线的总降幅; 7.58% 是固定新通信设置后, DG 对 torch 的额外降幅。两个比例分母不同, 体现通信与计算改动的消融, 不是两个独立可线性叠加百分数。

Q8: A2A 降低 81%, 为什么简历反而不突出它?

它主要来自把通信并行度从很低的 2 CTA 提高到实际 32 blocks, DG 独立贡献仅 1.67%, 且最优 median 点 P95 较差。AllGatherV 的 12.80% 和同设置 7.58% 更能准确表达我做联合控核的贡献, 避免将整个通信参数变化算给 GEMM。

Q9: 开两个 stream 就一定重叠吗?

不一定。数据依赖、CUDA 连接配置、事件、资源占用及提交顺序都可能造成串行。代理使用 MAX_CONNECTIONS=8 后, 还必须通过 Nsys 的 start/end 和实际 kernel grid 检查; 本项直接记录了候选 160/160 样本真实重叠, 而不是只看代码里有两个 stream。

Q10: 可以把 12 CTA + DG120 直接上线吗?

不能无条件照搬。当前为单机四卡、Torch 2.10/CUDA 12.8/NCCL 2.27.5; 原训练为 Torch 2.11/CUDA 13.1/NCCL 2.29 且存在八卡/跨节点差异。应复现真实通信量与并行拓扑, 检查正确性、实际 overlap、尾延迟及端到端 step, 再决定是否保留配置。

7. 追问与表达红线

- 最值得讲的工程判断: 敢于推翻“200 μ s 是异常”的最初假设, 以源码数据流和完整 range 证据修正问题定义。
- 不说“GPU 硬分区”“NCCL 独占 12 个 SM”“已经整网提速 12.8%”; 这些都超出现有证据。
- 不混淆 FC1 自己必须等待的 TP AllGather 与其他 stream 的独立通信。
- 不混用 CUDA Event 整段时间、Nsys kernel 时间、联合 span 和训练 step; 每个指标都要说清包含什么。
- 下一步复验建议是方案, 不是已完成事实: 真实八卡与跨节点、真实软件栈、足够稳态步数、同通信设置消融、loss/grad 检查。

8. 来源与可追溯文件

1. 北京, 当前结论与最终扫描: `/volume/pt-train/users/zhaoye/doc/M2问题分析/analysis_sink_fc1/sink_fc1_quick_analysis.md`。
2. 北京, shape 纠错与同生产 TE 复现: `/volume/pt-train/users/zhaoye/doc/M2问题分析/analysis_sink_fc1/sink_fc1_actual_shape_correction_zh.md`。
3. 北京, 完整竞争报告: `/volume/pt-train/users/zhaoye/doc/M2问题分析/analysis_sink_fc1/sink_fc1_nccl_sm_conflict_report_zh.md`。
4. 北京, 真实 TE 调用顺序: `/volume/pt-train/users/zhaoye/latent-moe-handoff/transformer-engine-d9b7fc5770a8/transformer_engine/pytorch/module/linear.py`。
5. 北京, 最终复验 CSV: `/volume/pt-train/users/zhaoye/doc/M2问题分析/analysis_sink_fc1/comm_repro/joint_grid_m8192_confirmed_summary.csv`。

6. 北京, 正确 shape 双 stream 代理: `/volume/pt-train/users/zhaoye/doc/M2问题分析/analysis_sink_fc1/comm_repro/benchmark_fc1_joint_grid_m8192_4gpu.py`。
7. 北京, 历史 M4096 结果 (仅供解释迭代): `/volume/pt-train/users/zhaoye/doc/M2问题分析/analysis_sink_fc1/comm_repro/joint_fc1_variable_comm_report_zh.md`。

本次重新连通上海 CPU 并检索其文档; 本项可复查的完整汇总归档在北京。生产栈匹配复现记录中的 Shanghai 执行脚本不改变这些文件的北京存放位置。